Short-Horizon Carbon Monoxide Forecasting with Environmental Sensor Data: A Chronological Evaluation of Five Regression Models

Bal Krishna
Department of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
bkbitbp123@gmail.com

Robin Prakash Mathur
Department of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab, India
mathur.robin@gmail.com

***Abstract*—Short-term forecasts of urban carbon monoxide (CO) can support exposure warnings and help interpret low-cost sensor networks. Their usefulness depends on whether model comparisons are made honestly on future data. This report studies one-hour-ahead CO prediction on the UCI Air Quality dataset. Holds hourly records from five metal-oxide sensors, temperature, humidity, and a co-located reference analyser. Five regression models are compared: multiple linear regression, k-nearest neighbours (KNN), linear support vector regression (SVR), Random Forest. Gradient Boosting, together with a persistence baseline that repeats the current CO reading. Predictors are the sensor channels, weather variables, calendar variables, the current CO value. Three lagged CO values; targets and lags are built only across consecutive hourly records. After filtering, 6,932 rows remain and are split chronologically into 4,852 training, 1,040 validation, and 1,040 test rows. Hyperparameters are selected on the validation block, models are refitted on training plus validation data, and each is evaluated once on the test block. Gradient Boosting (300 trees, depth 6) achieves the lowest test error (MAE 0.376 mg/m^3, RMSE 0.552 mg/m^3, $R^2 = 0.815$), with Random Forest close behind (RMSE 0.563 mg/m^3); both clearly outperform linear regression, linear SVR, KNN, and persistence. Errors are roughly twice as large for the highest-quartile CO hours and are largest around the evening forecast origins, so overall averages overstate performance when concentrations are high. The findings come from a single test period at one site and are not a general guarantee; limitations and a reproducibility procedure are documented.**

***Index Terms*—carbon monoxide, air quality, time-series forecasting, gradient boosting, random forest, support vector regression, chronological validation**

I. Introduction

Carbon monoxide is produced mainly by incomplete combustion, and road traffic is a major urban source. Short-term CO forecasts are useful because concentrations in a street canyon can change substantially within hours, following traffic and meteorological conditions. Reference-grade analysers are accurate but expensive, so there is considerable interest in combining cheaper sensors with statistical learning [2]. Such sensors, still, respond to several gases and to temperature and humidity, which makes their raw outputs hard to interpret directly and motivates data-driven models.

This project asks a practical question: *how well can the CO concentration one hour ahead be predicted from sensor, weather, calendar. Recent CO-history features, and how do five common regression models compare on a chronological held-out period?* The target is a continuous quantity, so the task is regression rather than classification, and performance is reported with MAE, RMSE, and the coefficient of determination R^2. An R^2 of 0.815 so means that about 81.5% of the test-set variance is explained; it is not a classification accuracy.

The contributions are modest and practical: (i) a documented preprocessing pipeline that builds next-hour targets and lags only across consecutive hourly records; (ii) a like-for-like comparison of five models plus a persistence baseline under one chronological protocol, with all tuning done on a separate validation block; (iii) an error audit covering higher-CO hours, forecast-origin hour of day. Signed bias; and (iv) an explicit list of limitations found on inspecting the use. The remainder of the report reviews related work (Section II), describes the data and target (Sections III–IV), the models and protocol (Sections V–VI), the results (Section VII), and discusses limitations and conclusions (Sections VIII–IX).

II. Related Work

A. Data and sensor calibration

The dataset used here was introduced by De Vito et al., who studied on-field calibration of a multisensor electronic nose against a co-located reference analyser in an urban setting [2]; the data are distributed through the UCI Machine Learning Repository [1]. That work concerns estimating a present concentration from sensor responses. Our setting is different: we forecast the next hour and treat sensor channels, weather, and CO history jointly as predictors.

B. Regression methods

The model families follow standard texts [14]–[16]. Linear regression supplies an interpretable baseline. KNN regression averages the responses of nearby training points and traces back to nearest-neighbour methods [10], [11]. Support vector regression fits a function within an ε-insensitive tube while

penalising large deviations [7], [8], building on the support-vector framework of Cortes and Vapnik [9]. Random Forests average many decorrelated trees grown on bootstrap samples [3], and their main tuning parameters have been examined empirically [13]. Gradient boosting builds an additive tree ensemble by stage-wise fitting of loss gradients [4], [5]; scalable variants such as XGBoost are widely used in applied prediction tasks [6], although the classical adoption is used here.

C. Evaluation of forecasts

For time-ordered data, random shuffling can place future observations in the training set and give optimistic estimates. Forecasting texts recommend evaluating on a later period than the one used for fitting and comparing against simple benchmarks such as the naive (persistence) forecast [17]. Studies of validation for time series discuss when ordinary cross-validation is acceptable and when ordered or blocked schemes are preferable [19]–[21]. The choice of error metric is also debated: RMSE and MAE emphasise different aspects of the error distribution, and reporting both is often advised [18], [22], [23]. This report follows these recommendations by using a chronological split, a persistence benchmark, and several complementary metrics.

III. Dataset and Problem Formulation

The UCI Air Quality dataset contains hourly averaged responses of five metal-oxide chemical sensors (labelled PT08.S1 to PT08.S5), temperature (T), relative humidity (RH), absolute humidity (AH). Reference-analyser concentrations of CO, non-methane hydrocarbons, benzene, NO_x, and NO_2, recorded in an Italian city from March 2004 to April 2005 [1], [2]. The source file is semicolon-delimited with comma decimal marks, and missing measurements are coded as -200. The repository describes 9,357 hourly instances [1]. Missing codes are frequent in some channels, which is why record eligibility and imputation are handled explicitly below. The chemical sensor channels are cross-sensitive: each responds to several gases and to temperature and humidity, so the weather variables are included as predictors rather than discarded [2].

Let y_t denote the reference CO concentration, CO(GT), at hour t in mg/m^3. The forecasting target is y_{t+1}, and the model receives information available at hour t. The reference analyser readings for non-methane hydrocarbons, benzene, NO_x. NO_2 are deliberately *not* used as predictors, because they would not necessarily be available at forecast time and would blur the sensor-based forecasting question.

IV. Preprocessing and Feature Engineering

All processing is implemented in R and run from a single script. The steps are as follows.

Import and cleaning. The file is read with a semicolon delimiter and comma decimal mark; empty trailing columns and blank rows are removed; all measurement columns are converted to numeric; and every -200 value is replaced by `NA`. Date and time are combined (format `%d/%m/%Y %H.%M.%S`, time zone set to UTC for parsing only) and records are sorted chronologically.

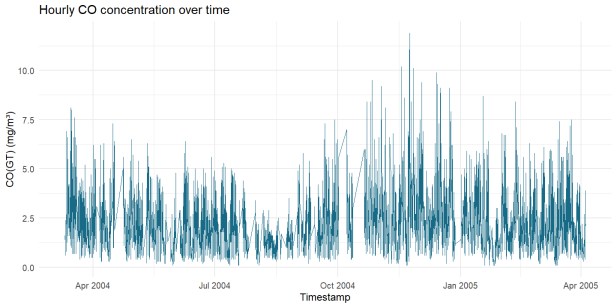


Fig. 1. Next-hour CO target for the eligible rows. Straight segments across the series join observations on either side of data gaps.

Calendar features. Hour of day, month, and day of week are derived from the timestamp and enter the models as integers.

Target and lag construction. The target y_{t+1} is assigned only when the next source record is exactly one hour after t and its CO value is observed. Lagged values y_{t-1}, y_{t-2}, and y_{t-3} (CO_Lag1–3) are populated only when every intervening step in the source timeline is a one-hour step. This prevents the lags from silently bridging data gaps.

Row filtering. Rows are retained only if the target, the current CO value y_t (CO_Current), and all three lags are present, which leaves 6,932 rows. consecutive rows of the *filtered* table are not always one hour apart (the script's gap printout, 0 exact one-hour gaps among 6,931 differences, is computed on the filtered table and is that's why not a measure of misalignment); the one-hour condition is enforced earlier, on the ordered source timeline because rows are dropped after the target and lags are built.

Predictors. The model input has 15 variables: the five sensor channels, T, RH, AH, hour, month, day of week, the current CO value y_t, and the three CO lags. The current CO value is a genuine predictor in all five machine-learning models, and it is also the forecast of the persistence baseline.

Missing predictors. Remaining missing values in sensor and weather predictors are imputed with the median of the fitting data. Medians, and for KNN and SVR the means and standard deviations used for standardisation, are computed on the fitting partition only and then applied unchanged to later partitions.

Fig. 1 shows the eligible target series and Fig. 2 its distribution. The series exhibits pronounced daily peaks, a few multi-day gaps, and a right-skewed distribution with occasional values above 10 mg/m^3.

V. Models

A. Persistence baseline

The baseline predicts $\hat{y}_{t+1} = y_t$. It takes no fitting and shows how much the learned models add beyond simply carrying the present reading forward [17].

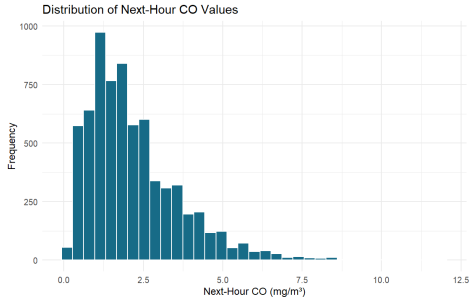


Fig. 2. Distribution of next-hour CO values among the 6,932 eligible rows.

B. Multiple linear regression

Ordinary least squares is fitted on all 15 predictors, $\hat{y}_{t+1} = \beta_0 + \sum_j \beta_j x_{j,t}$ [15]. It has no hyperparameters.

C. KNN regression

The prediction is the mean target of the k nearest training rows in standardised feature space (Euclidean distance) [10], [14]. The candidate values were $k \in \{5, 9, 15, 25\}$; $k = 25$ gave the lowest validation RMSE. Because the best value lies at the upper edge of the grid, a larger k might have performed slightly better; the grid was not extended. KNN is sensitive to the scale of each variable, which is why standardisation is applied.

D. Linear support vector regression

An ε-insensitive SVR with a linear kernel and $\varepsilon = 0.1$ is fitted on standardised predictors [7]. The cost parameter was chosen from $\{0.1, 1, 10\}$; $C = 10$ was selected, again the largest value in the grid. A linear kernel keeps the model close to regularised linear regression and was chosen for transparency; nonlinear kernels [8] were not evaluated.

E. Random Forest

The forest uses 300 trees and a minimum node size of 5, with the number of variables tried per split (m_{try}) chosen from $\{3, 5, 7\}$ [3], [12]. Validation selected $m_{\text{try}} = 7$. No scaling is needed for tree models.

F. Gradient Boosting

A Gaussian-loss boosted tree ensemble is fitted with the R `gbm` package, with a bag fraction of 0.8 and a minimum of 10 observations per terminal node [4]. A grid of 100 configurations was searched on the validation block: number of trees $\{200, 300, 500, 700, 1000\}$, interaction depth $\{2, 3, 4, 5, 6\}$, and shrinkage $\{0.01, 0.03, 0.05, 0.1\}$. The selected configuration has 300 trees and depth 6.

VI. Experimental Design

A. Chronological split

After filtering, rows are kept in time order and divided without shuffling: the first 70% (4,852 rows) for training, the next 15% (1,040 rows) for validation, and the final 15% (1,040 rows) for testing. The first and last forecast origins of the test block are 16 February 2005, 04:00 and 4 April 2005, 13:00; the target is the CO value one hour after each origin. Because rows without a valid target or lag history were removed, the test block is not perfectly contiguous: its origin timestamps contain 18 breaks in the hourly sequence.

TABLE I
Hyperparameter grids and settings selected on the validation block.

Model	Search space	Selected
Linear regression	none	—
KNN	$k \in \{5, 9, 15, 25\}$	$k = 25$
Linear SVR	$C \in \{0.1, 1, 10\}$	$C = 10$
Random Forest	$m_{\text{try}} \in \{3, 5, 7\}$	7
Gradient Boosting	trees, depth, shrinkage (100)	300, depth 6

TABLE II
Validation-block performance (1,040 rows) of models fitted on the training block. MAE and RMSE in mg/m^3.

Model	MAE	RMSE	R^2
Multiple Lin. Regression	0.5909	0.8323	0.7109
KNN ($k = 25$)	0.6467	0.9361	0.6344
Linear SVR ($C = 10$)	0.5263	0.7723	0.7511
Random Forest ($m_{\text{try}} = 7$)	0.5030	0.7239	0.7813
Gradient Boosting (300, depth 6)	0.4734	0.6836	0.8050

B. Tuning and final evaluation

Each candidate model is fitted on the training block and scored by RMSE on the validation block; the best setting per model is retained. The selected models are then refitted on the combined training and validation data, with medians and scaling statistics recomputed on that combined block, and evaluated once on the test block. The test block was not used to choose any hyperparameter.

C. Hyperparameter search and validation results

Table I summarises the search spaces and the selected settings, and Table II gives the corresponding validation-block errors for the models fitted on the training block only.

The validation ranking (Gradient Boosting, Random Forest, linear SVR, linear regression, KNN) is consistent with the test ranking except that linear SVR and linear regression swap places. Suggests that the two linear models are essentially tied. Validation errors are noticeably higher than test errors for every model, so the validation block appears to be the harder period.

TABLE III
TEST-SET PERFORMANCE (1,040 ROWS), SORTED BY RMSE. UNITS OF MAE AND RMSE ARE MG/M^3.

Model	MAE	RMSE	R^2
Gradient Boosting (300, depth 6)	0.3759	0.5517	0.8151
Random Forest ($m_{\text{try}} = 7$)	0.3927	0.5627	0.8077
Multiple Lin. Regression	0.4650	0.6645	0.7318
Linear SVR ($C = 10$)	0.4436	0.6714	0.7262
KNN ($k = 25$)	0.5215	0.7030	0.6998
Persistence (current CO)	0.5169	0.7957	0.6154

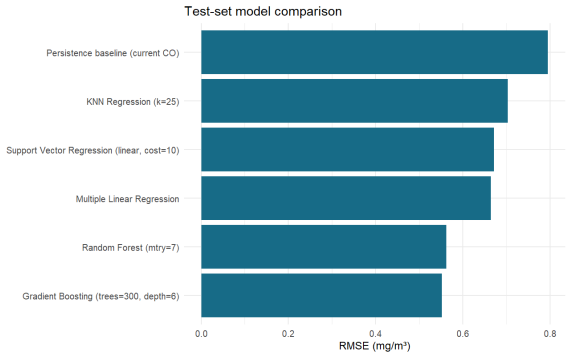


Fig. 3. Test-set RMSE of the five models and the persistence baseline.

D. Evaluation metrics

For n test rows with observed values y_i, predictions $\hat{y}_i$, and mean $\bar{y}$,

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \tag{1}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}, \tag{2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}. \tag{3}$$

MAE and RMSE are in mg/m^3. RMSE penalises large errors more heavily than MAE and is the primary ranking criterion here; MAE gives the typical absolute miss. R^2 compares the model with always predicting the test mean, and can in principle be negative [18]. For reference, the test targets have a mean of about 1.98 and a standard deviation of about 1.28 mg/m^3, so an RMSE of 0.55 corresponds to roughly 43% of the standard deviation. Signed mean error (bias), $\frac{1}{n}\sum_i(\hat{y}_i - y_i)$, is reported in addition to the three metrics above to show systematic over- or under-prediction [23].

VII. RESULTS

A. Model comparison

Table III reports test performance, and Fig. 3 shows the RMSE ranking.

Gradient Boosting ranks first on all three metrics. Its RMSE is about 2% lower than that of Random Forest and about 31% lower than that of the persistence baseline. Given one test period and a single run of each stochastic ensemble, the gap between the two tree ensembles is small and should not be over-interpreted; the clearer finding is that both ensembles separate from the linear and distance-based models by roughly 0.1 mg/m^3 of RMSE. Linear regression and linear SVR are close in RMSE. SVR has the lower MAE (0.444 vs. 0.465), consistent with its ε-insensitive loss being less dominated by large residuals. KNN, even after tuning, does not beat the persistence baseline on MAE (0.522 vs. 0.517) although it does on RMSE and R^2.

All learned models improve on persistence in RMSE and R^2, which indicates that sensor, weather, and calendar information adds value beyond the current reading. Validation RMSE values were higher than test values for the selected models (for example 0.684 vs. 0.552 for Gradient Boosting), so the validation block appears to have been harder than the test block; absolute errors should so be compared within a block, not across blocks.

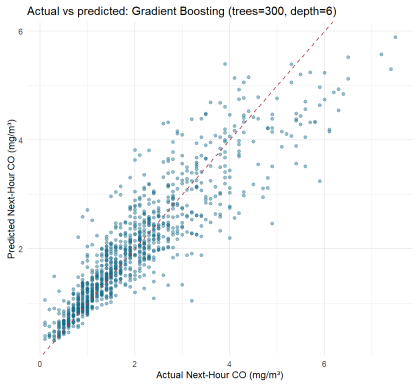


Fig. 4. Actual versus predicted next-hour CO for the Gradient Boosting model on the test block. The dashed line is the ideal one-to-one relation.

B. Predictions and variable importance

Fig. 4 plots actual against predicted values for the best model. Points follow the diagonal closely at low and moderate concentrations and scatter more at high concentrations.

Fig. 5 shows the Random Forest importance plot. The two panels use different measures and do not give identical orderings: permutation-based %IncMSE ranks hour of day first, followed by the current CO value and the first CO lag. IncNodePurity ranks the current CO value first, followed by the PT08.S2 and PT08.S1 sensor channels. Both measures place the current CO value among the most influential variables. Importance scores describe how this fitted forest uses the variables and do not build causal effects, especially because several predictors are correlated.

C. Bias and error by forecast-origin hour

Table IV lists the mean signed error on the test block. All models have small positive bias, between 0.009 and 0.097 mg/m^3; Gradient Boosting is the least biased overall. This hides a split: for the highest-quartile hours (defined on the test targets) its mean error is about -0.36 mg/m^3, whereas

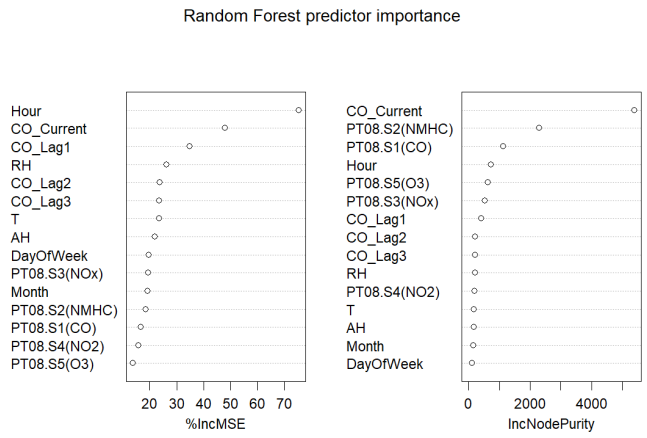


Fig. 5. Random Forest predictor importance (left: %IncMSE; right: IncNodePurity), from the model refitted on training plus validation data.

TABLE IV
MEAN SIGNED ERROR (PREDICTION − ACTUAL) ON THE TEST BLOCK, MG/M^3.

Model	Bias
Persistence (current CO)	0.027
Multiple Lin. Regression	0.097
KNN ($k = 25$)	0.071
Linear SVR ($C = 10$)	0.014
Random Forest ($m_{\text{try}} = 7$)	0.071
Gradient Boosting (300, depth 6)	0.009

for the remaining hours it is about $+0.15$ mg/m^3. The model because of this tends to under-predict peaks and slightly over-predict ordinary hours, a typical shrinkage effect of averaging-based learners.

Error also varies with the time of day at which the forecast is issued (Table V). Overnight origins are easy (MAE 0.145), while origins in the morning rush (06:00–09:59) and especially the evening (16:00–20:59) are hardest (MAE 0.492 and 0.589), when traffic-driven concentrations rise and fall quickly. Overall, 75.3% of Gradient Boosting predictions are within 0.5 mg/m^3 of the observed value and 91.4% within 1.0 mg/m^3.

D. High-CO error audit

Warning applications depend on the high end of the distribution, so the best model was also examined on test rows whose actual next-hour CO was at or above the 75th percentile of the training-plus-validation targets (Table VI).

The 192 higher-CO rows (18.5% of the test block) have an MAE about 2.0 times and an RMSE about 1.7 times the overall values. The overall metrics so overstate how well the model performs when concentrations are high. Is the situation where accuracy matters most for any alerting use.

VIII. DISCUSSION

The ranking suggests that nonlinear interactions, such as between time of day and recent CO history, carry useful signal beyond what a linear model captures. That tree ensembles handle the mixed scales and correlated predictors without extra preprocessing. The strong persistence baseline is also informative: hourly CO is autocorrelated, so much of the achievable skill comes from the current reading. The added value of the models is the remaining 30% or so of RMSE. This is consistent with the general advice that any forecasting model should first be compared with a naive benchmark [17].

TABLE V
GRADIENT BOOSTING TEST ERROR BY FORECAST-ORIGIN HOUR (MG/M^3).

Origin hours	n	MAE	RMSE
00–05	229	0.145	0.200
06–09	152	0.492	0.675
10–15	283	0.314	0.437
16–20	235	0.589	0.776
21–23	141	0.396	0.551

TABLE VI
ERROR AUDIT OF THE GRADIENT BOOSTING MODEL ON THE TEST BLOCK.

Segment	n	MAE	RMSE
All test observations	1,040	0.3759	0.5517
Higher-CO observations	192	0.7675	0.9452

The relation between metrics deserves comment. RMSE and MAE can rank models differently because RMSE weights large misses more heavily; here the two agree for the top two models but disagree for linear SVR versus linear regression, and for KNN versus persistence. Neither metric is uniformly preferable [22], [23], which is why Table III reports both along with R^2. The bias results show that low average bias can coexist with systematic under-prediction of peaks, so a single summary number is insufficient for a warning application.

The importance plot suggests that the current CO reading, hour of day. The first CO lag are among the variables the forest relies on most, which fits the intuition that both persistence and the daily traffic cycle matter. Whether the five sensor channels add information beyond CO history was not tested by ablation, so no claim is made about their marginal value.

A. Limitations and threats to validity

- **Single period and site.** All results come from one seven-week test block at one monitoring location, with no repeated or rolling-origin evaluation, so uncertainty in the model ranking is not quantified.
- **Boundary overlap.** The target of the last row of one block is the CO value y_t of the first row of the next block when those hours are adjacent. The overlap involves at most one value per boundary and was not removed; a gap between blocks would be a stricter design.
- **Gradient Boosting refit.** Shrinkage was searched on the validation block. The final refit in the script fixes shrinkage at 0.05 rather than reusing the validation-selected value, and the selected shrinkage is not printed in the output. The reported Gradient Boosting result is hence for the printed tree count and depth with shrinkage 0.05.

- **Tuning cost and seeds.** The 100-configuration search is large relative to the other models. Gives Gradient Boosting more opportunity to fit the validation block. No random seed is set in the script, so Random Forest and Gradient Boosting results can vary slightly between runs.
- **Imputation and sensor drift.** Median imputation is simple. Metal-oxide sensors are known to drift over time; the models were not tested on data from later in the sensor's life or from other sites.
- **Point forecasts only.** No prediction intervals are produced, and only a one-hour horizon is studied.

Chronological ordering and training-only preprocessing reduce the main routes by which future information could reach the models, but this report does not claim that leakage is impossible.

B. Reproducibility

The project is run from the project root in RStudio with `source("scripts/run_project.R")`, with the dataset at `data/AirQualityUCI.csv`. The script installs any missing packages (`readr`, `dplyr`, `tidyr`, `ggplot2`, `e1071`, `randomForest`, `gbm`, `FNN`) and writes validation and test metrics, test predictions, the high-CO audit, and the five figures to `outputs/`. All numbers in this report are taken from the generated `test_metrics.csv` and `high_pollution_error_audit.csv`.

IX. Conclusion and Future Work

This report compared five regression models and a persistence baseline for forecasting next-hour CO on the UCI Air Quality data under a strictly chronological protocol with training-only preprocessing and validation-based tuning. Gradient Boosting (300 trees, depth 6) gave the best held-out performance, with MAE 0.376 mg/m^3, RMSE 0.552 mg/m^3. R^2 of 0.815; Random Forest was a close second, and both ensembles outperformed linear regression, linear SVR, KNN, and persistence in RMSE and R^2. About three quarters of the Gradient Boosting predictions fell within 0.5 mg/m^3 of the observed value.

Three cautions accompany these results. First, error roughly doubles for the highest-quartile CO hours. The best model tends to under-predict peaks, so the overall averages overstate performance where it matters most for warnings. Second, errors are concentrated around the morning and evening forecast origins. Third, the evidence comes from one site and one test period of about seven weeks, with no repeated seeds or rolling origins. The blending has documented issues (a fixed shrinkage in the final refit, no random seed, and a possible one-value overlap at block boundaries).

Future work should (i) use rolling-origin evaluation with multiple seeds and a gap between blocks; (ii) report prediction intervals through quantile regression or conformal methods; (iii) run ablations to measure the contribution of the sensor channels and of each lag; (iv) extend the horizon beyond one hour and compare recurrent networks such as LSTMs [24] and scalable boosting [6]; and (v) test transfer to other sites and to later sensor periods to address drift. Within these limits, the study offers a transparent and reproducible baseline for sensor-based short-horizon CO forecasting.

References

[1] S. De Vito, "Air Quality," UCI Machine Learning Repository, 2008. [Online]. Available: https://doi.org/10.24432/C59K5F

[2] S. De Vito, E. Massera, M. Piga, L. Martinotto, and G. Di Francia, "On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario," *Sensors and Actuators B: Chemical*, vol. 129, no. 2, pp. 750–757, 2008.

[3] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[5] A. Natekin and A. Knoll, "Gradient boosting machines, a tutorial," *Frontiers in Neurorobotics*, vol. 7, art. 21, 2013.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[7] H. Drucker, C. J. C. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support vector regression machines," in *Advances in Neural Information Processing Systems 9*, 1997, pp. 155–161.

[8] A. J. Smola and B. Schölkopf, "A tutorial on support vector regression," *Statistics and Computing*, vol. 14, no. 3, pp. 199–222, 2004.

[9] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[10] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[11] N. S. Altman, "An introduction to kernel and nearest-neighbor nonparametric regression," *The American Statistician*, vol. 46, no. 3, pp. 175–185, 1992.

[12] A. Liaw and M. Wiener, "Classification and regression by randomForest," *R News*, vol. 2, no. 3, pp. 18–22, 2002.

[13] P. Probst, M. N. Wright, and A.-L. Boulesteix, "Hyperparameters and tuning strategies for random forest," *WIREs Data Mining and Knowledge Discovery*, vol. 9, no. 3, art. e1301, 2019.

[14] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.

[15] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning*. New York, NY, USA: Springer, 2013.

[16] M. Kuhn and K. Johnson, *Applied Predictive Modeling*. New York, NY, USA: Springer, 2013.

[17] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[18] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

[19] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[20] C. Bergmeir, R. J. Hyndman, and B. Koo, "A note on the validity of cross-validation for evaluating autoregressive time series prediction," *Computational Statistics & Data Analysis*, vol. 120, pp. 70–83, 2018.

[21] V. Cerqueira, L. Torgo, and I. Mozetič, "Evaluating time series forecasting models: An empirical study on performance estimation methods," *Machine Learning*, vol. 109, pp. 1997–2028, 2020.

[22] C. J. Willmott and K. Matsuura, "Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance," *Climate Research*, vol. 30, no. 1, pp. 79–82, 2005.

[23] T. Chai and R. R. Draxler, "Root mean square error (RMSE) or mean absolute error (MAE)? Arguments against avoiding RMSE in the literature," *Geoscientific Model Development*, vol. 7, no. 3, pp. 1247–1250, 2014.

[24] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.